

INSTITUTO SUPERIOR TECNOLÓGICO DE TURISMO Y PATRIMONIO "YAVIRAC"

YaviBot

Manual Técnico del sistema de Certificado de Matrícula,
trámites automatizados y verificación de documentos
por código QR.

AUTOR

Andrew Carrera

TUTORA

Elizabeth Mera

VERSIÓN DEL DOCUMENTO

1.0

FECHA

Agosto 2026

Descripción general del sistema

YaviBot es un asistente conversacional para estudiantes y docentes del Instituto Yavirac, construido como aplicación móvil híbrida (Ionic/Angular) con un backend de automatización (n8n) que actúa como API completa sobre una base de datos PostgreSQL compartida con el resto del sistema institucional.

El sistema resuelve, de principio a fin y sin intervención de un panel administrativo dedicado, cuatro trámites: emisión de Certificado de Matrícula con verificación pública por código QR, Anulación de Matrícula, Reseteo de Contraseña del correo institucional, y Reporte de Incidencias de Laboratorio (para el rol docente).

La identidad del usuario se resuelve con un único flujo, sin pantalla de login: cédula de identidad seguida de un código de verificación de un solo uso (OTP) enviado al correo institucional. El mismo flujo distingue automáticamente si quien lo usa es un estudiante o un docente, y adapta el menú de opciones que ve.

Principio rector del proyecto — n8n no consume una API: n8n es la API. No existe un servidor Node/Express/NestJS intermedio. Cada trámite es un *workflow* de n8n con un nodo Webhook como punto de entrada; toda la lógica de negocio —validaciones, generación de códigos únicos, reglas de idempotencia— vive dentro de esos workflows.

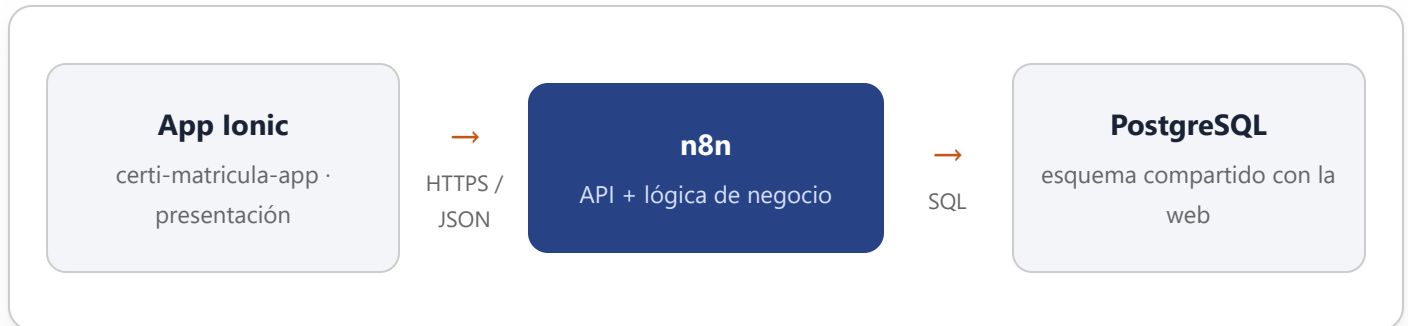
Alcance dentro del proyecto de titulación

Este manual documenta la **aplicación móvil** del proyecto de titulación "*Desarrollo de una aplicación web y móvil para automatizar los procesos de mesa de ayuda mediante la herramienta de orquestación n8n en el Instituto Superior Tecnológico Yavirac*". El objetivo general del proyecto es optimizar los procesos administrativos del instituto reduciendo los tiempos de atención y seguimiento de requerimientos; esta app cubre específicamente los trámites de **Estudiante** y el rol **Docente** descritos en la sección Trámites del sistema. La gestión administrativa (paneles de Secretaría/TI, seguimiento de tickets, asignación manual) corresponde a la aplicación web del

proyecto, construida de forma independiente sobre la misma base de datos institucional — ver Trazabilidad de requerimientos para el detalle de qué cubre cada aplicación.

Arquitectura general

Tres capas, con una responsabilidad clara y sin lógica de negocio filtrándose entre ellas.



Capa 1 — Presentación

La app Ionic/Angular (`certi-matricula-app/`) no contiene reglas de negocio: solo llama webhooks y renderiza lo que reciben. La identidad del usuario, el menú disponible y el estado de la conversación viven en un único componente de chat conversacional (`chat.page.ts`) que orquesta cada trámite como una máquina de estados.

Capa 2 — API y lógica de negocio

Cada archivo en `n8n-backend/workflows/` es, a la vez, la definición de un endpoint HTTP y la implementación completa de su lógica: validaciones, verificación de sesión OTP, generación de códigos únicos, envío de correos, idempotencia. No existe una capa de código separada — los nodos **Webhook**, **Postgres**, **IF** y **Code** de cada workflow son, literalmente, el backend.

Capa 3 — Datos

PostgreSQL 16 con la extensión `pgcrypto` habilitada. El esquema es **real y compartido** con el sistema web/panel administrativo del instituto (~20 tablas en total); esta app únicamente lee y escribe el subconjunto que necesita (ver Modelo de datos). Si el esquema cambia, solo se ajustan los nodos Postgres de los workflows afectados — el contrato hacia la app no cambia.

Por qué importa esta separación — permite que dos equipos (esta app y el sistema web) construyan de forma independiente sobre la misma base de datos, sin coordinar despliegues, mientras respeten

las mismas restricciones de integridad (claves únicas, estados válidos) definidas a nivel de esquema.

Stack tecnológico

Frontend — aplicación móvil

COMPONENTE	TECNOLOGÍA	ROL
Framework UI	@ionic/angular 8	Componentes híbridos, apariencia nativa
Framework de aplicación	@angular/core 20	Componentes, servicios, inyección de dependencias
Runtime nativo	@capacitor/core 8	Empaquetado iOS/Android, plugin de estado de red
Lenguaje	TypeScript 5.9	Tipado estático en toda la app
Reactividad	RxJS 7.8	Observables para las llamadas HTTP a n8n
Generación de PDF	jspdf	Certificado de matrícula con membrete oficial, generado en el navegador
Generación de QR	qrcode	Código QR del certificado, a partir de la URL de verificación
Verificación humana	Google reCAPTCHA v2	Casilla de verificación en el único paso sin OTP previo

Backend — automatización

COMPONENTE	TECNOLOGÍA	ROL
Motor de workflows / API	n8n (self-hosted)	Único backend — cada workflow es un endpoint
Base de datos	PostgreSQL 16 + pgcrypto	Esquema institucional compartido
Identidad institucional	Google Workspace Admin SDK	Reseteo real de contraseña de correo
Envío de correo	SMTP (mesa de ayuda)	OTP, confirmaciones, avisos a responsables

COMPONENTE	TECNOLOGÍA	ROL
Orquestación local	Docker + Docker Compose	Contenedores de n8n y PostgreSQL para desarrollo

Modelo de datos

El esquema real tiene cerca de 20 tablas para el sistema institucional completo (chatbot + panel administrativo + alertas de laboratorio). YaviBot solo toca el siguiente subconjunto:

TABLA	USO	PARA QUÉ
estudiantes	Lectura	Identificar al estudiante por cédula
docentes	Lectura	Segunda identidad — mismo flujo cédula + OTP
carreras / periodos_academicos	Lectura (JOIN)	Nombre de carrera y periodo lectivo vigente
otp_codigos	Lectura/escritura	Código de verificación hashado (SHA-256 vía pgcrypto) y prueba de sesión reciente
qr_codigos	Lectura/escritura	Identificador único (UUID) del certificado, base del QR
certificados	Lectura/escritura	Certificado de matrícula emitido
tickets / tipos_solicitud	Lectura/escritura	Trámites manuales (Anulación de Matrícula)
asignaciones_responsables / eventos	Lectura/escritura	A quién se notifica cada trámite nuevo, y auditoría
laboratorios / alertas / alerta_historial	Lectura/escritura	Catálogo e incidencia reportada por un docente
adjuntos	Lectura/escritura (opcional)	Metadatos de la foto adjunta a una incidencia (el binario va a disco, no a la BD)
usuarios_panel / roles	Solo lectura	Resolver a qué persona real corresponde un responsable

Decisiones de modelado relevantes

- **El OTP nunca se guarda en texto plano** — se hashea con SHA-256 (pgcrypto) tanto al generarlo como al verificarlo.
- **El código único del certificado (codigoUnico) es un UUID real** (gen_random_uuid()), con restricción UNIQUE a nivel de base de datos — un QR duplicado es estructuralmente imposible, sin importar qué cliente escriba.
- **Idempotencia forzada en dos niveles:** el workflow verifica antes de insertar (para responder con datos existentes en vez de un error), y además **certificados** tiene `UNIQUE(estudiante_id, periodo_lectivo_codigo)` — cierra la condición de carrera si dos solicitudes llegan casi al mismo tiempo.
- **Reemisión de certificado sin duplicar QR:** si el estudiante vuelve a pedir su certificado del mismo periodo, se actualiza `fecha/hora` sobre la misma fila (mismo `qr_id` de siempre) — el QR impreso nunca cambia, pero la fecha mostrada, tanto en el PDF como al escanearlo, siempre refleja la última emisión.
- **Estados de tickets en español** (Pendiente / En Proceso / Resuelto), traducidos por los workflows a `EN_PROCESO / COMPLETADO` para el contrato que consume la app.

Especificación de la API

10 workflows con Webhook, autenticados con cabecera X-Api-Key. Formato de respuesta plano (sin envelope): éxito → 200 con el objeto/arreglo directo; error de negocio → 4xx con { "error": "mensaje" }.

Dos capas de autenticación — la X-Api-Key filtra tráfico que no pasa por la app, pero es la misma para todos los usuarios. La protección real contra *"conozco la cédula de alguien y quiero actuar en su nombre"* es la **sesión OTP**: los endpoints que modifican datos o exponen información personal exigen, del lado del servidor, un `otp_codigos.usado = true` de los últimos 20 minutos para esa cédula.

5.1 — Identidad y verificación

POST /consultar-estudiante reCAPTCHA v2

Busca la cédula en **estudiantes**; si no hay match, busca en **docentes**. Responde siempre **200** — null si no existe, o el objeto con `tipoUsuario`: "ESTUDIANTE" | "DOCENTE". Único paso sin OTP previo, protegido en su lugar con reCAPTCHA v2 contra enumeración de cédulas.

REQUEST

```
{ "cedula": "0102030405" }
```

RESPONSE 200

```
{ "tipoUsuario": "ESTUDIANTE",  
  "nombres": "...", "carrera": "...", ... }
```

POST /enviar-ticket-verificacion

Genera un código de 6 dígitos, lo guarda hasheado con expiración de 10 minutos y lo envía al correo institucional del usuario identificado.

- **Cooldown de 60s por cédula** — este paso no tiene CAPTCHA propio.
- Responde correoEnmascarado (ej. an***@yavirac.edu.ec), nunca el código.

POST /verificar-ticket

Compara el código recibido (hasheado) contra `otp_codigos`. La app solo muestra el menú cuando responde `{ "valido": true }`.

- **Límite de 5 intentos fallidos en 10 minutos** por cédula — bloquea con 429 al superarlo.

5.2 — Certificado de matrícula

POST /generar-certificado-matricula Sesión OTP Automático

Único trámite **100% automático** del menú. Crea el código único (UUID) y el registro del certificado; si ya existía uno para ese estudiante y periodo, **actualiza su fecha** en vez de duplicar el QR.

REQUEST

```
{ "cedula": "0102030405" }
```

RESPONSE 200

```
{ "codigoUnico": "744c...",  
  "fechaEmision": "...", "urlVerificacion":  
  "..." }
```

400 Cédula no matriculada en el periodo actual.

403 Sin sesión OTP válida reciente.

POST

/enviar-certificado-pdf

Sesión OTP

La app genera el PDF real (membrete oficial + QR) en el navegador con `certificado-pdf.service.ts`, lo convierte a base64 y lo envía aquí para que n8n lo adjunte y lo mande por correo institucional. El workflow verifica que `codigoUnico` pertenezca de verdad a esa `cedula` antes de enviar nada.

POST

/verificar-certificado

Público — sin login

El QR impreso codifica una URL pública (`/verificar-certificado/{codigo}`) que cualquiera puede escanear para validar autenticidad — funciona como firma digital de Secretaría. No exige OTP a propósito.

- La cédula viaja **siempre enmascarada** (175****314).
- `nivel`, `carrera`, `modalidad` y `periodoActual` se leen **en vivo**; `fechaEmision` refleja la última reemisión.
- Código inexistente → 404 con `autentico: false`, nunca un 500.

5.3 — Trámites de estudiante

POST

/crear-ticket-solicitud

Sesión OTP

Registra Anulación de Matrícula como ticket manual (`tipoTramite: "ANULACION_MATRICULA"`), resuelto por personal administrativo fuera de esta app. Asigna responsable automáticamente y le envía un aviso por correo.

- **Idempotente**: si ya existe un ticket *activo* del mismo tipo, devuelve el existente en vez de duplicarlo — forzado también por restricción única en base de datos.

POST`/resetear-contrasena-correo`

Sesión OTP

Automático

A diferencia de los demás trámites manuales, este es **100% automático**: genera una contraseña temporal segura y la aplica de inmediato contra **Google Workspace Admin SDK**. La contraseña nueva viaja únicamente por correo institucional, nunca en la respuesta HTTP ni en auditoría.

200 Reseteo ejecutado — contraseña enviada por correo.

403 Sin sesión OTP reciente.

429 Cooldown de 15 min entre resets exitosos.

502 Falló el proveedor de identidad — el estudiante reintenta desde el chat.

5.4 — Rol docente

POST`/consultar-laboratorios`

Catálogo fijo de laboratorios, usado para poblar el paso de selección al reportar una incidencia.

POST`/reportar-incidencia-laboratorio`

Sesión OTP

El backend **vuelve a validar** del lado del servidor que la cédula sea de un docente real — no confía en que la app ya lo decidió. Foto adjunta opcional (JPEG/PNG, máx. 5MB), guardada en disco, nunca como binario en la base de datos.

Medidas de seguridad implementadas

Cada control responde a una amenaza concreta identificada durante el desarrollo, no a una lista genérica de buenas prácticas.

- ☐ **Autenticación de webhook (X-API-Key)** — filtra tráfico que no pasa por la app antes de ejecutar cualquier lógica.
- ☐ **reCAPTCHA v2** en /consultar-estudiante, el único paso sin OTP previo — contra enumeración de cédulas.
- ☐ **Sesión OTP verificada server-side** (no solo en la app) en los 5 endpoints que actúan en nombre de un estudiante/docente específico — ventana de 20 minutos.
- ☐ **Cooldown de 60s** al reenviar el código de verificación, y **límite de 5 intentos en 10 min** al verificarlo — contra fuerza bruta y abuso de envío.
- ☐ **Idempotencia forzada a nivel de base de datos** (restricciones UNIQUE) además del chequeo del workflow — cierra condiciones de carrera entre esta app y el sistema web.
- ☐ **Enmascaramiento de datos sensibles** — correo (an***@...) y cédula (175****314) en las respuestas donde no hace falta el dato completo.
- ☐ **Contraseña nueva solo por correo** — nunca en la respuesta HTTP ni en el log de auditoría del reseteo.
- ☐ **Validación server-side redundante** — ej. reportar-incidencia-laboratorio reconfirma que la cédula sea de un docente real, sin confiar en la app.
- ☐ **Auditoría de eventos** (tabla eventos) para trámites creados, asignados y reseteos ejecutados.

Trámites del sistema

TRÁMITE	ROL	EJECUCIÓN	GENERA DOCUMENTO
Certificado de Matrícula	Estudiante	Automático	QR + correo, PDF adjunto
Anulación de Matrícula	Estudiante	Manual (ticket)	—
Reseteo de contraseña institucional	Estudiante	Automático	—
Reportar incidencia de laboratorio	Docente	Manual (revisión)	Foto opcional adjunta

Reseteo de contraseña: por qué es automático y no un ticket

Requerimiento explícito de la coordinación académica: el estudiante, ya autenticado por cédula + OTP, resetea su contraseña al instante, sin intervención humana. La prueba de que pasó por el OTP se reutiliza de `otp_codigos.usado` (ventana de 20 min) en vez de crear un mecanismo de sesión nuevo; el reseteo real se ejecuta contra Google Workspace Admin SDK con una credencial de Service Account de privilegio mínimo (rol "Reset user passwords", no Super Admin).

Identidad dual: estudiante / docente

Segunda identidad de usuario agregada sin tocar el flujo de identificación existente ni crear una pantalla de login separada.

1. POST `/consultar-estudiante` busca primero en `estudiantes`; si no hay match, busca en `docentes`. La respuesta incluye el discriminador `tipoUsuario`.
2. POST `/enviar-ticket-verificacion` hace la misma búsqueda dual para saber a qué correo enviar el OTP.
3. POST `/verificar-ticket` no distingue rol — valida el OTP contra `otp_codigos` por cédula únicamente.
4. La app guarda el `Usuario` identificado (unión discriminada `Estudiante | Docente`) y elige el menú según `tipoUsuario`: el estudiante ve sus 3 trámites; el docente ve únicamente **Reportar incidencia en laboratorio**.

Foto de incidencia — se envía como base64 desde `<input type="file" capture="environment">` (sin plugin nativo de cámara), el workflow la escribe en disco dentro del volumen Docker `uploads_data` y solo entonces registra la fila en `adjuntos`. Adjuntar foto es siempre opcional.

Estructura del proyecto

Frontend — certi-matricula-app/

```
src/app/  
  chat/ // componente conversacional – orquesta todos los trámites  
  verificar-certificado/ // página pública de verificación por QR  
  services/ // estudiante.service.ts, certificado-pdf.service.ts  
  interceptors/ // ApiKeyInterceptor – agrega X-API-Key a cada request  
  models/ // estudiante.model.ts – tipos e interfaces del dominio  
  utils/ // validación de cédula ecuatoriana, etc.  
src/environments/ // environment.ts / environment.prod.ts
```

Backend — n8n-backend/

```
workflows/ // 10 workflows .json, uno por endpoint, importables en n8n  
docker-compose.yml // contenedores n8n + PostgreSQL  
init.sql // esquema + datos de prueba para desarrollo local  
ARQUITECTURA.md · CONTRATO-API.md · ESQUEMA-BD.md // documentación técnica de referencia
```

Instalación y ejecución local

Backend (n8n + PostgreSQL)

1. Levantar los contenedores: `docker compose up -d` desde `n8n-backend/` (crea las tablas de `init.sql` en el primer arranque).
2. Abrir n8n (`http://localhost:5678`) e importar los 10 workflows (*Workflows* → *Import from File*).
3. En cada nodo Postgres, configurar la credencial real de base de datos (placeholder REEMPLAZAR).
4. En cada nodo Webhook, configurar la credencial Header Auth (X-API-Key) — mismo valor que `environment.apiKey` en la app.
5. En los nodos de envío de correo, configurar la credencial SMTP.
6. Poner la Secret Key de reCAPTCHA v2 en el nodo "Verificar CAPTCHA (Google)".
7. Activar los 10 workflows.

Frontend (Ionic/Angular)

1. `npm install` dentro de `certi-matricula-app/`.
2. Confirmar en `environment.ts`: `usarMock: false` y `n8nBaseUrl: 'http://localhost:5678/webhook'`.
3. `npx ng serve` — la app queda disponible en `http://localhost:4200`.

Nota — la app también puede correr con `environment.usarMock = true`, sin backend real, contra datos simulados que respetan el mismo contrato — útil para desarrollar la interfaz sin depender de n8n.

Trazabilidad de requerimientos

Mapeo entre los requerimientos funcionales (RF), no funcionales (RNF) y casos de uso (CU) del proyecto de titulación, y lo que efectivamente implementa esta aplicación móvil.

Requerimientos funcionales

CÓDIGO	REQUERIMIENTO	ESTADO EN ESTA APP
RF-01	Iniciar sesión	Implementado — cédula + código OTP por correo institucional
RF-02	Recuperar contraseña	Implementado — /resetear-contrasena-correo, 100% automático
RF-09	Solicitar certificado de matrícula	Implementado — /generar-certificado-matricula + QR
RF-12	Registrar incidencias de laboratorio	Implementado — /reportar-incidencia-laboratorio (rol Docente)
RF-13	Generar ticket automáticamente	Implementado — código correlativo generado por Postgres tras el insert
RF-17	Notificaciones automáticas	Implementado — correo de confirmación al usuario y aviso al responsable asignado
RF-18	Generar certificados en PDF	Implementado — generado en el cliente (jsPDF) con membrete oficial y QR
RF-21	Automatizar procesos con n8n	Implementado — n8n es la API completa de esta app, no solo un motor auxiliar

Requerimientos no funcionales (selección)

CÓDIGO	REQUERIMIENTO	CÓMO SE CUMPLE EN ESTA APP
RNF-01	Seguridad	reCAPTCHA v2 + sesión OTP verificada server-side en cada acción sensible (ver Seguridad)

CÓDIGO	REQUERIMIENTO	CÓMO SE CUMPLE EN ESTA APP
RNF-09	Integridad de datos	Restricciones UNIQUE a nivel de base de datos para idempotencia, además de la validación del workflow
RNF-14	Auditoría	Tabla eventos — registra creación/asignación de tickets y resets ejecutados
RNF-15	Confidencialidad	Enmascarado de correo y cédula en las respuestas públicas o de bajo contexto
RNF-16	Automatización	Los 10 workflows corren sin intervención manual una vez configurados

Casos de uso (selección)

CÓDIGO	CASO DE USO	ESTADO EN ESTA APP
CU-01	Iniciar sesión	Implementado — identidad por cédula + OTP
CU-02	Registrar ticket	Implementado — Anulación de Matrícula, Incidencia de Laboratorio
CU-04	Recibir notificaciones	Implementado — por correo institucional en cada trámite

Anexos

Glosario

OTP	One-Time Password — código de un solo uso enviado por correo para verificar identidad.
Webhook	Nodo de n8n que expone un workflow como endpoint HTTP.
Idempotencia	Propiedad por la cual repetir la misma operación no produce resultados duplicados.
Trámite automático	Se ejecuta y resuelve sin intervención humana (Certificado, Reseteo de contraseña).
Trámite manual	Queda registrado como ticket para revisión de personal administrativo (Anulación, Incidencia).

Documentación técnica de referencia

Este manual consolida la documentación detallada que vive en `n8n-backend/`:

`ARQUITECTURA.md` (diseño y seguridad completos), `CONTRATO-API.md` (especificación exacta de cada endpoint), `ESQUEMA-BD.md` (definición completa de tablas) y `ARRANQUE-LOCAL.md` (guía paso a paso de instalación).